Charts

Charts overview | UI components

Learn about the available chart components for visualizing data in UI extensions.

Use charts to display data visualizations in UI extensions. HubSpot provides two chart components: **BarChart** and **LineChart**. Both components use the same API, but the each type is better suited for different types of data. For example, a bar chart is generally recommended for comparing categorical data, while a line chart is recommended for time series plots or visualizing trends. To see an example of how to implement charts in a UI extension, check out **HubSpot's Charts example projects**. Note that this is a version `2025.1` project, but the chart component implementation would be similar for `2025.2`.

Learn more about creating custom charts with UI extensions by watching **this video** on the HubSpot Developers YouTube channel.

On this page:

Formatting data

Configuring axes

Stacking

Chart options

Colors

Design guidelines

```
import { BarChart, LineChart } from "@hubspot/ui-extensions"
```

For both types of charts, there are three main props:

`options` : an object that specifies options for the chart, such as showing data labels and tooltips. Learn more about **chart options**.

Formatting data

Data should be provided to a chart component in an array of objects containing key-value pairs, matching the following format `{string: number | string}` . Data will be displayed in the order it's provided to the component, so you will need to sort data beforehand if necessary. For example, to display data over time in a `LineChart` , you should sort the data in ascending/descending order of your `datetime` axis field before passing it to the chart component.

```
[
  {
    type: "referral",
    count: 35,
    location: "location_A",
  },
  {
    type: "direct",
    count: 12,
    location: "location_B",
  },
];
```

When building out a chart, keep the following in mind:

A chart can only graph one dataset, so you'll need multiple charts if you're working with multiple datasets.

For performance and readability, it's recommended for a chart to include no more than a few hundred entries, depending on the data. When working with larger datasets, it's important to consider the information you want to convey with the chart. You'll likely encounter issues with visual clarity before you encounter

because it introduces a secondary level of data in the `type` field.

```
[
  {
    type: {
      value: "referral",
      subType: "subtypeValue",
    },
    count: 35,
    location: "location_A",
  },
];
```

datetime values

For charts that include datetime data, you can use the following formats:

- Unix millisecond timestamp (`1758791725`)
- ISO 8601 timestamp (`2025-09-25T09:15:25+0000`)
- ISO 8601 date (`2024-09-25`)

Data options

It's recommended to pre-format your data into human-readable text so that it doesn't need any additional relabeling. However, if there are times when you can't pre-format certain values in your data, you can include an `options` field in the `data` prop to set `propertyLabels` . Including `options` will slightly change the way you format `data` :

- When only including a dataset array, you'll format the `data` prop as `data={dataArray}` .
- When including `options` , you'll need to format the `data` prop as an object containing both `data` and `options` fields as shown below. In `options` , you'll include a `propertyLabels` object, which then contains an object for each field and labels for each value.

HubSpot Developers

```
  data={{
    data: dealCountSample,
    options: {
      propertyLabels: {
        dealstage: {
          appointmentScheduled: 'Appointments scheduled (Sales)',
          closedWon: 'Closed won (Sales)',
        },
        user_id: {
          '194784': 'Sales user A',
          '295834': 'Sales user B',
          '938453': 'Sales user C',
        },
      },
    },
  }}
  axes={{
    x: {
      field: 'dealstage',
      fieldType: 'category',
      label: 'Deal Stage',
    },
    y: {
      field: 'count',
      fieldType: 'linear',
      label: 'Count of Deals',
     },
    options: { groupFieldByColor: 'user_id', stacking: true },
  }}
  options={{
    showLegend: true,
    showDataLabels: true,
    showTooltips: true,
  }}
  />
);
```

value will be used as the axis label, but you can also include a `label` parameter to set it separately.

> **Please note:**
>
> One axis must have a `fieldType` of `linear`.

```
<BarChart
  axes={{
    x: { field: "Product", fieldType: "category" },
    y: { field: "Amount", fieldType: "linear", label: "# of sales" },
  }}
/>;
```

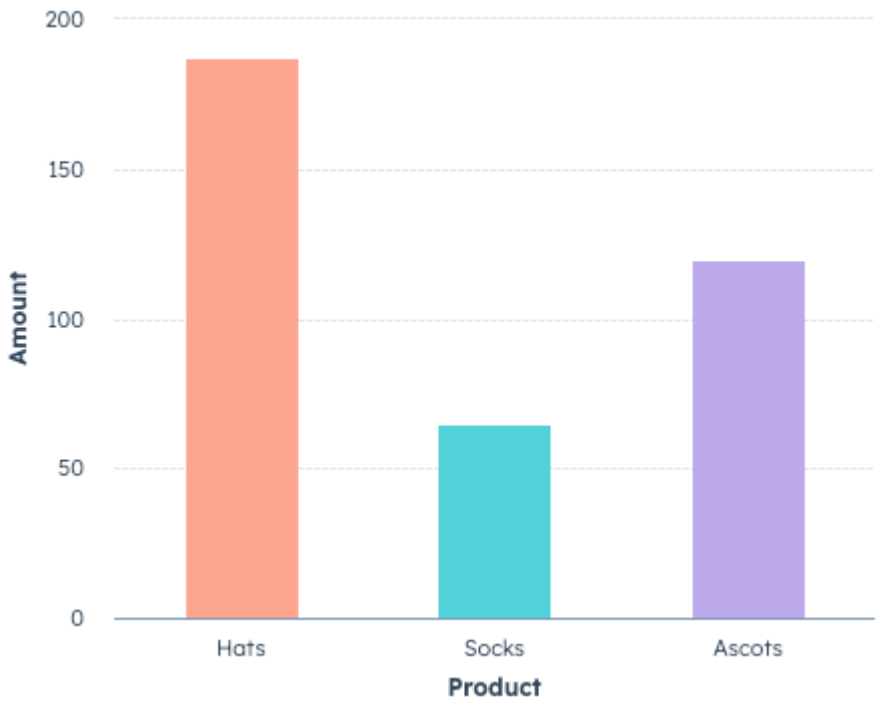
Parameter	Type	Description
`field`	String	The name of the field from the dataset.
`fieldType`	String	The type of data in the field. You can specify one of the following types: `category`: data that can be bucketed into categories or types, such as different types of products. `datetime`: date and time data. Charts support the following datetime formats: JavaScript timestamp (`1758791725`) ISO 8601 timestamp (`2025-09-25T09:15:25+0000`) ISO 8601 date (`2024-09-25`) `linear`: numerical data, such as quantity. One of the axes must have this `fieldType`.
`label`	String	The label to display on the axis. If not specified, the `field` value will be used instead.
`options`	Object	Additional configuration options for the axes:

HubSpot Developers

›

data will be stacked. Default is `false` .

The following bar chart displays sales data by type of product and count of sales. To add visual clarity, each bar is assigned a different **color** via the `groupFieldByColor` parameter in `options` .



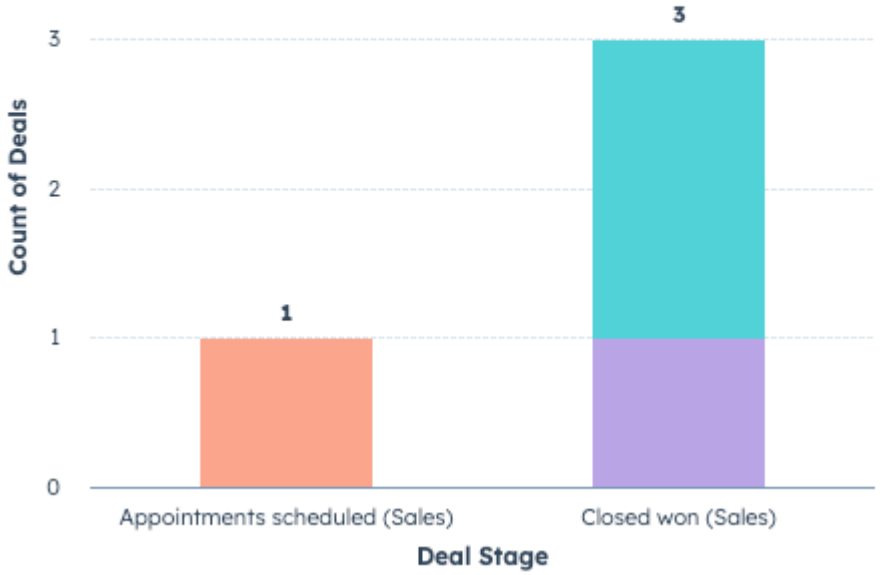
HubSpot Developers

```
  },
  {
    Product: "Socks",
    Amount: 65,
  },
  {
    Product: "Ascots",
    Amount: 120,
  },
];
return (
  <BarChart
    data={dailyInventorySample}
    axes={{
      x: { field: "Product", fieldType: "category" },
      y: { field: "Amount", fieldType: "linear" },
      options: {
        groupFieldByColor: "Product",
      },
    }}
  />
);
```

Stacking

Use the `stacking` axes option to stack data by group. For example, the following bar chart displays the number of deals by deal stage. The data also includes the sales rep who owns each deal. To visually distinguish sales reps in each column, `stacking` has been set to `true` .

HubSpot Developers



HubSpot Developers

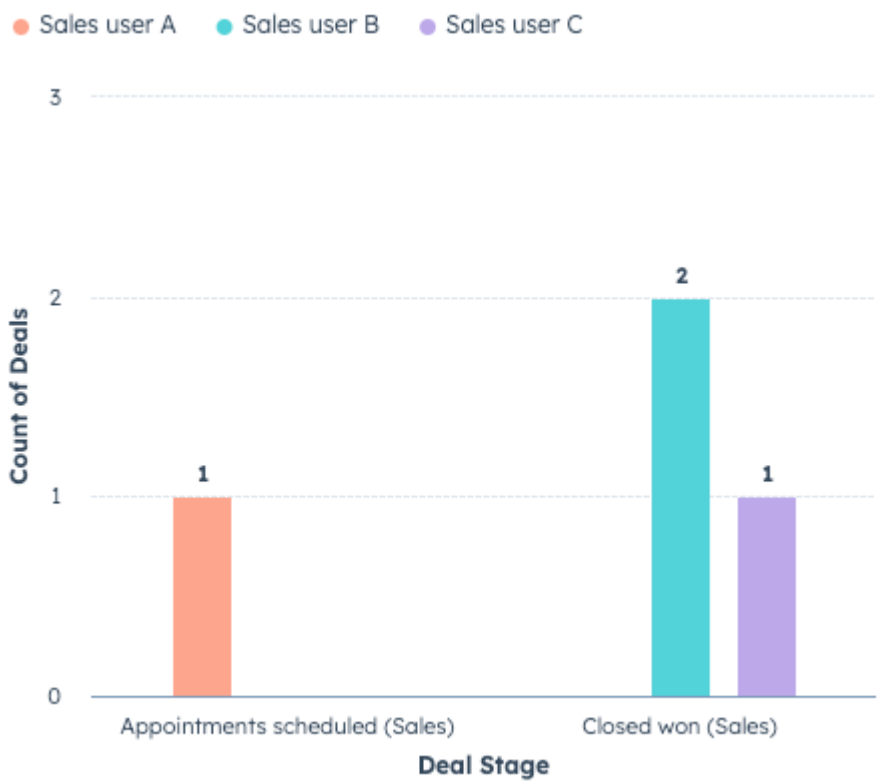
```
      >
        dealstage: "appointmentScheduled",
        user_id: "194784",
      },
      {
        count: 2,
        dealstage: "closedWon",
        user_id: "295834",
      },
      {
        count: 1,
        dealstage: "closedWon",
        user_id: "938453",
      },
    ];

    return (
      <BarChart
        data={{
          data: dealCountSample,
          options: {
            propertyLabels: {
              dealstage: {
                appointmentScheduled: "Appointments scheduled (Sales)",
                closedWon: "Closed won (Sales)",
              },
              user_id: {
                194784: "Sales user A",
                295834: "Sales user B",
                938453: "Sales user C",
              },
            },
          },
        }}
        axes={{
          x: {
            field: "dealstage",
            fieldType: "category",
            label: "Deal Stage",
          },
```

HubSpot Developers

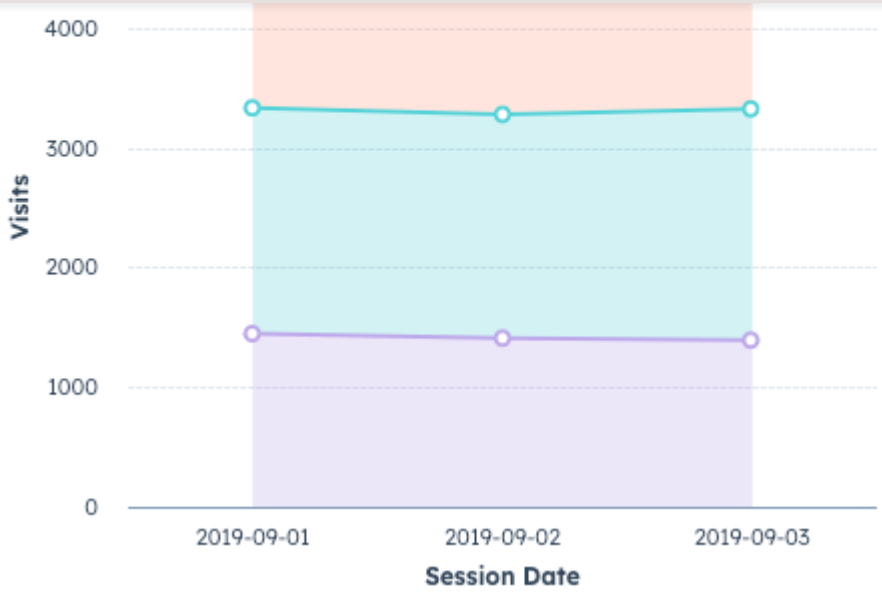
```
        >
          options: { groupFieldByColor: "user_id", stacking: true },
        }}
        options={{
          showLegend: true,
          showDataLabels: true,
          showTooltips: true,
        }}
      />
    );
  };
```

Compare the above visualization to the version below without stacking:



Similarly, you can use stacking in line charts, as shown in the example chart below. This chart measures website visits by date, broken down by source. Stacking in this example helps to emphasize volume over time.

HubSpot Developers



HubSpot Developers


```
    Visits: 1277,
  },
  {
    "Session Date": "2019-09-01",
    Breakdown: "Referrals",
    Visits: 1882,
  },
  {
    "Session Date": "2019-09-01",
    Breakdown: "Email",
    Visits: 1448,
  },
  {
    "Session Date": "2019-09-02",
    Breakdown: "Direct",
    Visits: 1299,
  },
  {
    "Session Date": "2019-09-02",
    Breakdown: "Referrals",
    Visits: 1869,
  },
  {
    "Session Date": "2019-09-02",
    Breakdown: "Email",
    Visits: 1408,
  },
  {
    "Session Date": "2019-09-03",
    Breakdown: "Direct",
    Visits: 1357,
  },
  {
    "Session Date": "2019-09-03",
    Breakdown: "Referrals",
    Visits: 1931,
  },
  {
    "Session Date": "2019-09-03",
```

HubSpot Developers

```
return (
  <LineChart
    data={visitsPerSourceOverTime}
    axes={{
      x: { field: "Session Date", fieldType: "category" },
      y: { field: "Visits", fieldType: "linear" },
      options: { groupFieldByColor: "Breakdown", stacking: true },
    }}
    options={{ showLegend: true }}
  />
);
```

Compare the above visualization to the version below without stacking:

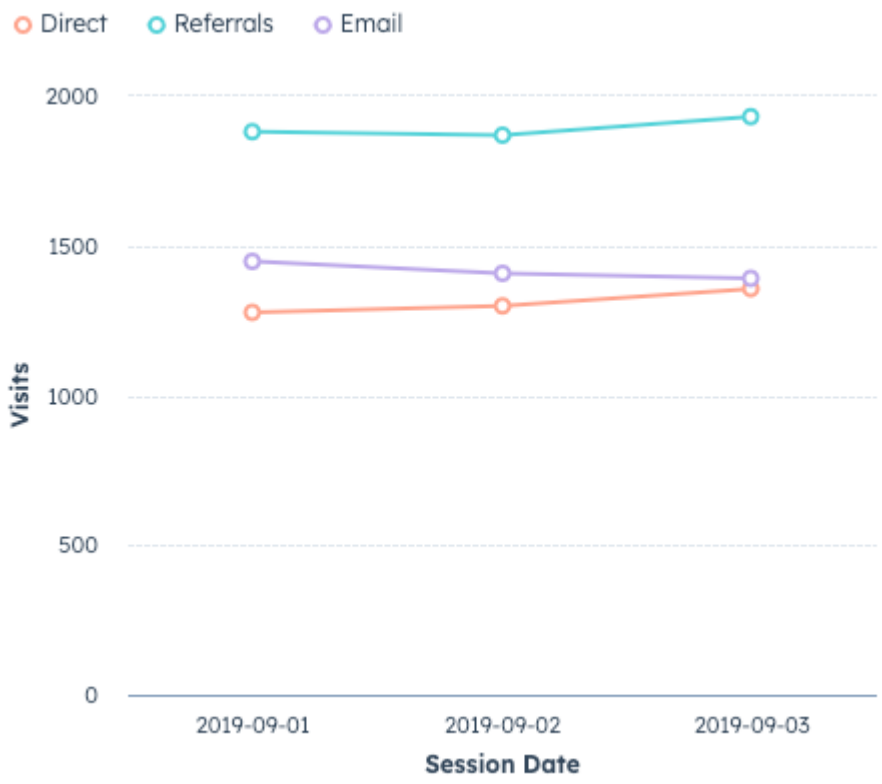
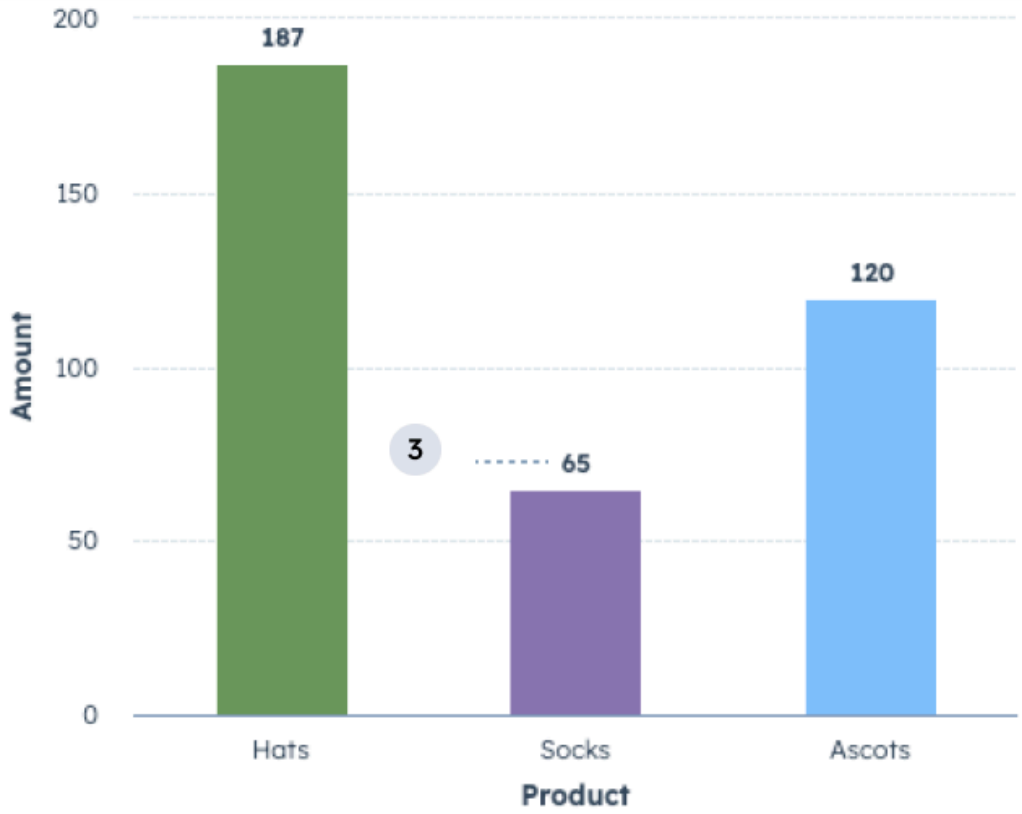


Chart options

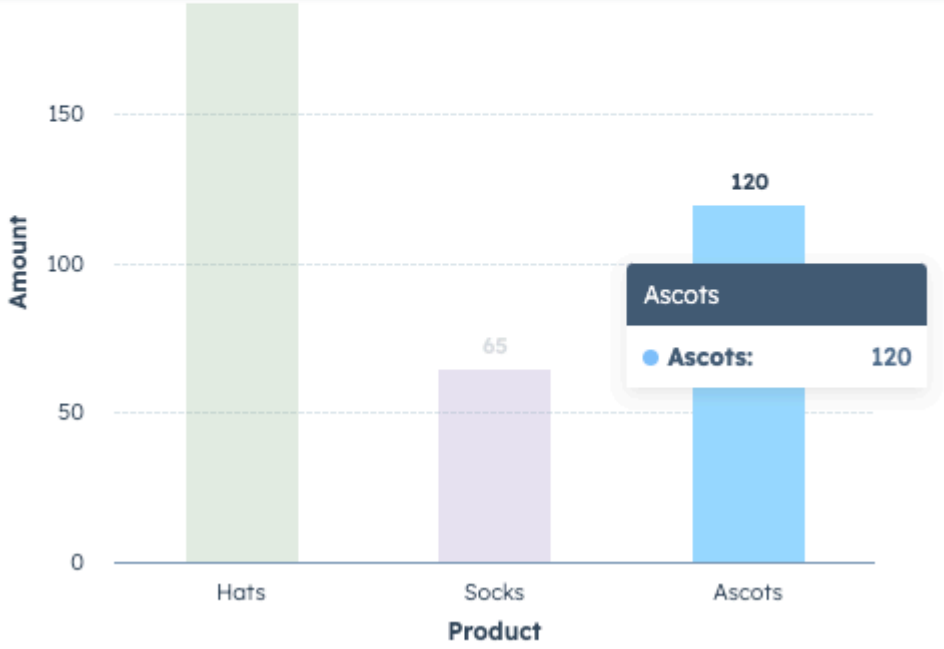
Using the `options` prop, you can configure a chart with options such as displaying a chart title, legend, data labels, or specifying the color list.

HubSpot Developers



1. **Title:** the title of the chart.
2. **Legend:** lists the data categories with their corresponding color for readability.
3. **Data labels:** labels for data points.
4. **Tooltips:** displays details for data points on hover.

HubSpot Developers




```
<BarChart
  data={dailyInventorySample}
  axes={{
    x: { field: "Product", fieldType: "category" },
    y: { field: "Amount", fieldType: "linear" },
    options: { groupFieldByColor: "Product" },
  }}
  options={{
    title: "My Chart",
    showTooltips: true,
    showLegend: true,
    showDataLabels: true,
    colorList: ["darkGreen", "darkPurple", "blue"],
  }}
/>;
```

Parameter	Type	Description
`title`	String	The title of the chart.
`showTooltips`	Boolean	When set to `true`, displays tooltips for data points on hover. Default is `false`.

HubSpot Developers

›

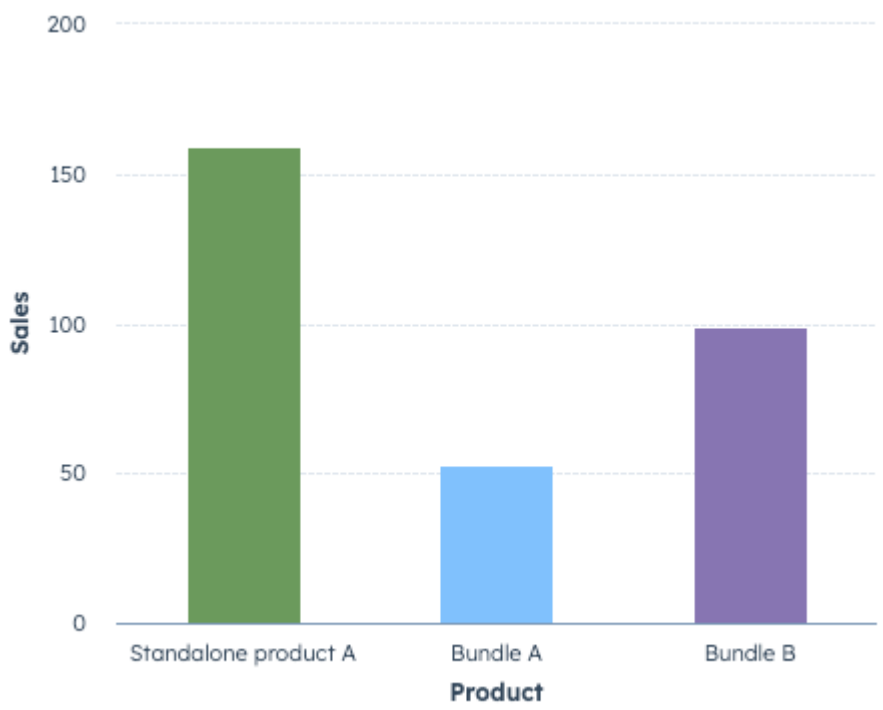
		for readability. Default is `false` .
`colorList`	Array	An array of strings specifying the order that colors should be used in the chart. Learn more about **colors**.

Colors

By default, HubSpot will apply colors to chart bars or lines using a default set of colors when `groupFieldByColor` is specified in axes `options` . You can customize these colors in two ways:

- To customize the order of colors selected by HubSpot, include the `colorList` field in the top-level `options` prop, then specify the colors you want to prioritize.
- To apply colors to specific values of the field specified in `groupFieldByColor` , include the `colors` field within the axes `options` .

For example, the chart below is configured to apply colors to data in the `Product` field using `groupFieldByColor` . The three colors (`darkGreen` , `blue` , `darkPurple`) will be applied first, then the standard color order will be applied to any additional bars.

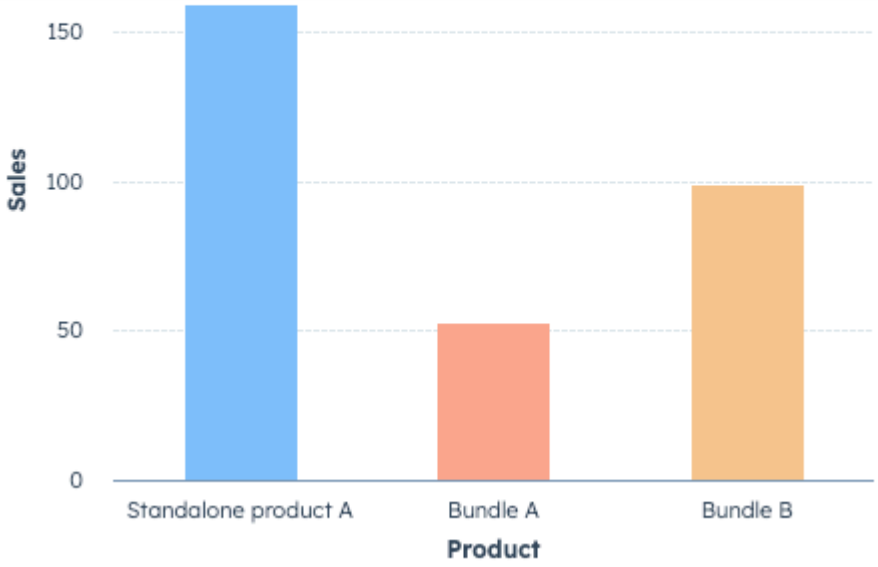


HubSpot Developers

```
  },
  {
    Product: "Bundle A",
    Sales: 53,
  },
  {
    Product: "Bundle B",
    Sales: 99,
  },
];
return (
  <BarChart
    data={dailyInventorySample}
    axes={{
      x: { field: "Product", fieldType: "category" },
      y: { field: "Amount", fieldType: "linear" },
      options: { groupFieldByColor: "Product" },
    }}
    options={{
      colorList: ["darkGreen", "blue", "darkPurple"],
    }}
  />
);
```

If instead you want to manually apply colors to specific field values, rather than have HubSpot assign colors in order, you could instead include the `colors` field in the axes `options`. In colors, you'll need to specify each value from the `groupFieldByColor` field to assign a color to.

HubSpot Developers



HubSpot Developers

```
  },
  {
    Product: "Bundle A",
    Sales: 53,
  },
  {
    Product: "Bundle B",
    Sales: 99,
  },
];
return (
  <BarChart
    data={dailyInventorySample}
    axes={{
      x: { field: "Product", fieldType: "category" },
      y: { field: "Sales", fieldType: "linear" },
      options: {
        groupFieldByColor: "Product",
        colors: {
          "Standalone product A": "blue",
          "Bundle A": "orange",
          "Bundle B": "yellow",
        },
      },
    }}
  />
);
```

Default color set

Below are the available colors in their default order.

Color	Hex value	Swatch
`orange`	#fea58e	
`aqua`	#51d3d9	

HubSpot Developers

›

pink	#ea90b1	
blue	#81c1fd	
green	#a4d398	
darkOrange	#c3705c	
darkAqua	#009ca2	
darkPurple	#8775b2	
darkYellow	#bb915b	
darkPink	#b05c7d	
darkBlue	#468cc4	
darkGreen	#6b9a5b	

Design guidelines

DO: title your data categories with human-readable text so they are easy to understand.

DO: use sentence-casing for the data categories and chart title (only first letter capitalized).

DO: sort your data in ascending/descending order of your x-axis field to prevent unordered rendering prior to passing it to a charting component. If you intend to display information over time (such as in a LineChart), your data will be displayed in the order you provide it.

middle column. Avoid using charts with many data points on smaller surfaces such as the preview panel or sidebar.

DON'T: use more than 14 data categories unless it cannot be avoided for your use case.

DON'T: use the same colors to indicate different data categories.

Last modified on January 28, 2026

Was this page helpful? Yes No

Table design patterns
‹ Previous

BarChart | UI components
Next ›

HubSpot Developers
Copyright © 2026 HubSpot, Inc
Privacy Policy
Security

Website Accessibility

Manage Cookies